

Advanced Machine Learning

DATA 642

Lecture 1 — Machine Learning Basics

*Notes and figures are from the book Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition. by Aurélien Géron. Publisher(s): O'Reilly Media, Inc. (Chapter 1)

1. Types of Machine Learning Systems

Supervised learning

In supervised learning, the training data you feed to the algorithm includes the desired solutions, called labels.

A typical supervised learning task is classification. The spam filter is a good example of this: it is trained with many example emails along with their class (spam or ham), and it must learn how to classify new emails.

Another typical task is to predict a target numeric value, such as the price of a car, given a set of features (mileage, age, brand, etc.) called predictors. This sort of task is called regression. To train the system, you need to give it many examples of cars, including both their predictors and their labels (i.e., their prices).

Here are some of the most important supervised learning algorithms

- k -Nearest Neighbors
- Linear Regression
- Logistic Regression
- Support Vector Machines (SVMs)
- Decision Trees and Random Forests
- Neural Networks

Unsupervised learning

In unsupervised learning the training data is unlabeled.

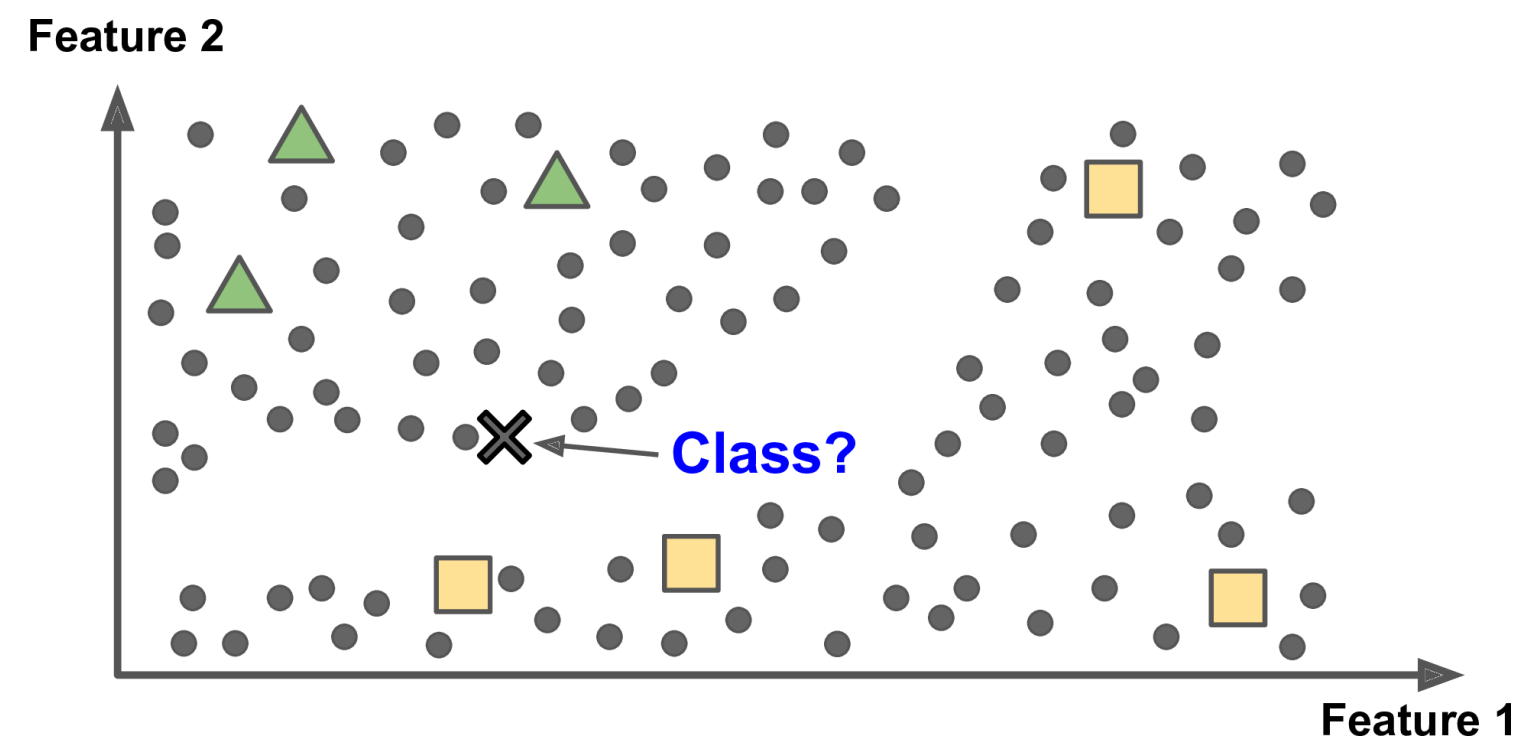
Here are some of the most important unsupervised learning algorithms

- Clustering
 - K -Means
 - Hierarchical Cluster Analysis (HCA)
- Anomaly detection and novelty detection
 - One-class SVM
- Dimensionality reduction and latent variable modeling
 - Principal Component Analysis (PCA)
 - Kernel PCA
 - t-distributed Stochastic Neighbor Embedding (t-SNE)
 - Independent component analysis
 - Tensor factorization
- Association rule learning
 - Apriori
 - Eclat

Semi-supervised learning

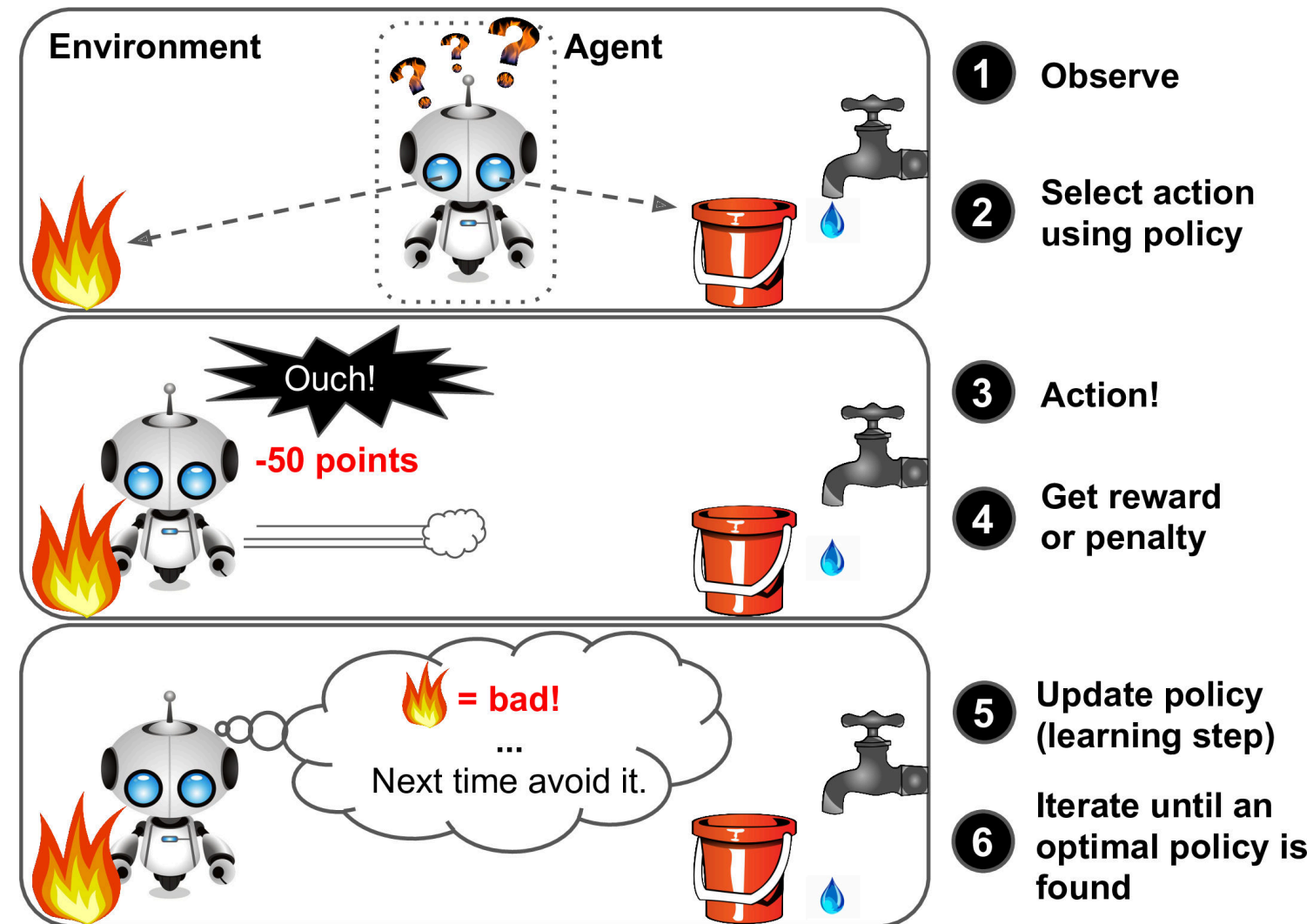
Some algorithms can deal with partially labeled training data, usually a lot of unlabeled data and a little bit of labeled data. This is called semi-supervised learning.

Some photo-hosting services, such as Google Photos, are good examples of this. Once you upload all your family photos to the service, it automatically recognizes that the same person A shows up in photos 1, 5, and 11, while another person B shows up in photos 2, 5, and 7. This is the unsupervised part of the algorithm (clustering). Now all the system needs is for you to tell it who these people are. Just one label per person and it is able to name everyone in every photo, which is useful for searching photos.



Reinforcement Learning

Reinforcement Learning is something very different. The learning system, called an agent in this context, can observe the environment, select and perform actions, and get rewards in return (or penalties in the form of negative rewards). It must then learn by itself what is the best strategy, called a policy, to get the most reward over time. A policy defines what action the agent should choose when it is in a given situation.



2. Batch and Online Learning

Another criterion used to classify machine learning systems is whether or not the system can learn incrementally from a stream of incoming data.

Batch learning

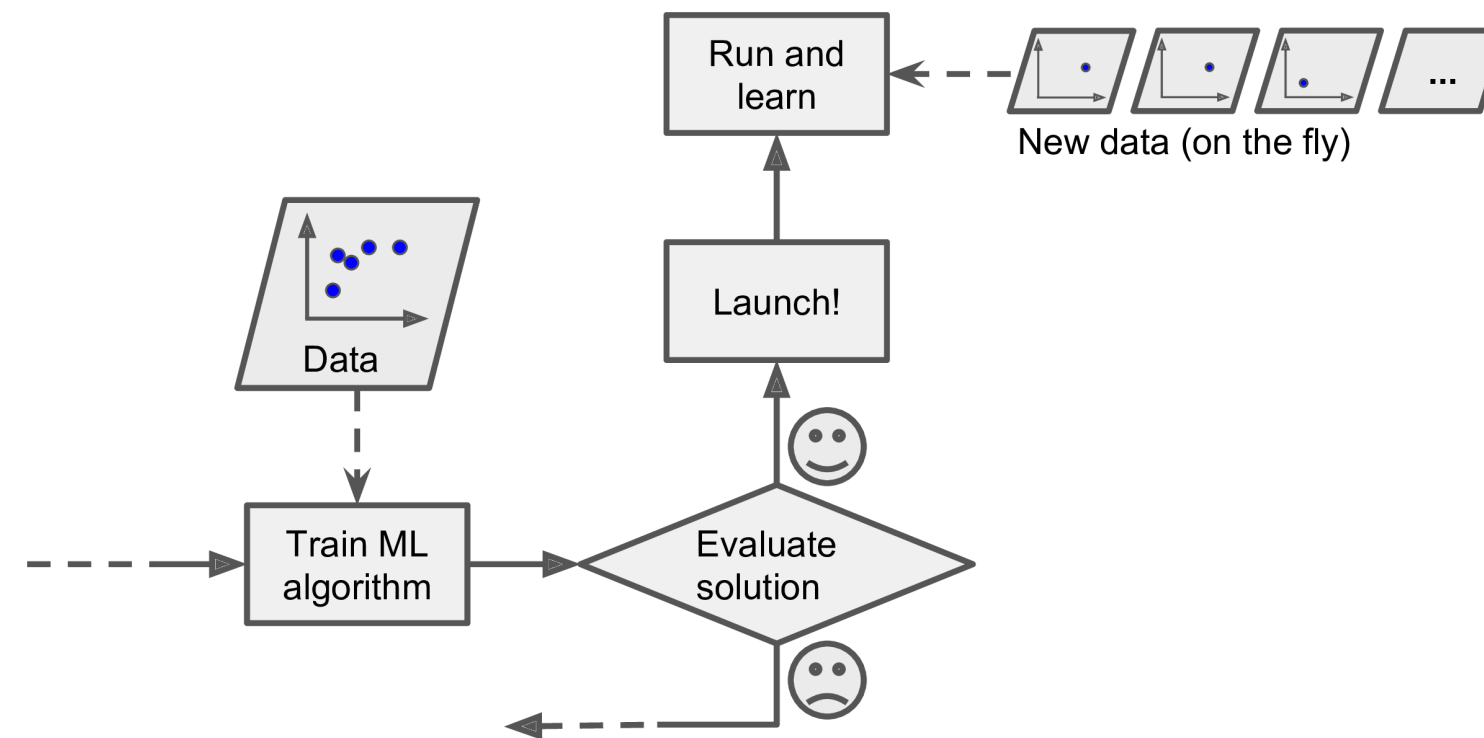
In batch learning, the system is incapable of learning incrementally: it must be trained using all the available data. This will generally take a lot of time and computing resources, so it is typically done offline. First the system is trained, and then it is launched into production and runs without learning anymore; it just applies what it has learned. This is called offline learning.

- The whole process of training, evaluating, and launching a machine learning system can be automated fairly easily, so even a batch learning system can adapt to change. Simply update the data and train a new version of the system from scratch as often as needed.
- This solution is simple and often works fine, but training using the full set of data can take many hours, so you would typically train a new system only every 24 hours or even just weekly. If your system needs to adapt to rapidly changing data (e.g., to predict stock prices), then you need a more reactive solution.

- Also, training on the full set of data requires a lot of computing resources (CPU, memory space, disk space, disk I/O, network I/O, etc.). If you have a lot of data and you automate your system to train from scratch every day, it will end up costing you a lot of money. If the amount of data is huge, it may even be impossible to use a batch learning algorithm.
- Finally, if your system needs to be able to learn autonomously and it has limited resources (e.g., a smartphone application or a rover on Mars), then carrying around large amounts of training data and taking up a lot of resources to train for hours every day is a showstopper.

Online learning

Online learning, involves updating the model continuously as new data points arrive. It adapts to changes in the data distribution over time and is suited for real-time and dynamic environments.



One important parameter of online learning systems is how fast they should adapt to changing data, i.e. learning rate.

- If you set a high learning rate, then your system will rapidly adapt to new data, but it will also tend to quickly forget the old data (you don't want a spam filter to flag only the latest kinds of spam it was shown).
- Conversely, if you set a low learning rate, the system will have more inertia; that is, it will learn more slowly, but it will also be less sensitive to noise in the new data or to sequences of non-representative data points (outliers).

Pros of online learning:

- Adaptability to changing data over time.
- Suitable for real-time and dynamic environments.
- Doesn't require holding the entire dataset in memory.

Cons of online learning:

- The model might become sensitive to the order of data points.
- Prone to noisy data, as the model might be affected by outliers or errors in the new data.
- Continuous updates can introduce challenges in managing model stability and convergence.

If we are talking about a live system, your clients will notice. For example, bad data could come from a malfunctioning sensor on a robot, or from someone spamming a search engine to try to rank high in search. To reduce this risk, you need to monitor your system closely and promptly switch learning off (and possibly revert to a previously working state) if you detect a drop in performance. You may also want to monitor the input data and react to abnormal data (e.g., using an anomaly detection algorithm).

3. Main Challenges of Machine Learning

Insufficient Quantity of Training Data

Data matters more than algorithms for complex problems was further popularized by Peter Norvig et al. in a paper titled "The Unreasonable Effectiveness of Data" published in 2009. It should be noted, however, that small- and medium sized datasets are still very common, and it is not always easy or cheap to get extra training data, so do not abandon algorithms just yet.

Nonrepresentative Training Data

It is crucial to use a training set that is representative of the cases you want to generalize to. This is often harder than it sounds: if the sample is too small, you will have sampling noise (i.e., nonrepresentative data as a result of chance), but even very large samples can be nonrepresentative if the sampling method is flawed. This is called sampling bias.

Poor-Quality Data

Obviously, if your training data is full of errors, outliers, and noise (e.g., due to poor-quality measurements), it will make it harder for the system to detect the underlying patterns, so your system is less likely to perform well. It is often well worth the effort to spend time cleaning up your training data.

Irrelevant Features

Your system will only be capable of learning if the training data contains enough relevant features and not too many irrelevant ones. Feature engineering includes

- Feature selection: selecting the most useful features to train on among existing features.
- Feature extraction: combining existing features to produce a more useful one (as we saw earlier, dimensionality reduction algorithms can help).
- Creating new features by gathering new data.

Underfitting the Training Data

Underfitting is the opposite of overfitting: it occurs when your model is too simple to learn the underlying structure of the data. For example, a linear model of life satisfaction is prone to underfit; reality is just more complex than the model, so its predictions are bound to be inaccurate, even on the training examples. The main options to fix this problem are:

- Selecting a more powerful model, with more parameters
- Feeding better features to the learning algorithm (feature engineering)
- Reducing the constraints on the model (e.g., reducing the regularization hyperparameter)

Overfitting the Training Data

Overfitting happens when the model is too complex relative to the amount and noisiness of the training data. The possible solutions are:

- To simplify the model by selecting one with fewer parameters (e.g., a linear model rather than a high-degree polynomial model), by reducing the number of attributes in the training data or by constraining the model.
- To gather more training data
- To reduce the noise in the training data (e.g., fix data errors and remove outliers)

